

MODEL 980A COMPUTER ASSEMBLY LANGUAGE CODING CONVENTIONS

MANUAL NO. 961961-9732

REISSUED 15 FEBRUARY 1974

NATIONAL BIOMEDICAL RESEARCH FOUNDATION
Georgetown University Medical Center
3900 Reservoir Road, N. W.
Washington, D. C. 20007

NOV 05 1974



TEXAS INSTRUMENTS
INCORPORATED

Copyright 1974
By
Texas Instruments Incorporated

All Rights Reserved

PRINTED IN U.S.A.

The information and/or drawings set forth in this document and all rights in and to inventions disclosed herein and patents which might be granted thereon disclosing or employing the materials, methods, techniques or apparatus described herein are the exclusive property of Texas Instruments Incorporated.

No disclosure of the information or drawings shall be made to any other person or organization without the prior consent of Texas Instruments Incorporated.

ENGINEERING CHANGE NOTICE

DATE	ECN	LEVEL	DESCRIPTION
2/2/73	373973	E	CHANGE PAGES: COVER, 2-6, 2-7, 2-8, 2-12, 2-15, 2-18, 2-19, 2-20, 2-21, 2-23, 2-24 & 2-28
8/18/73	380747		CHANGE PAGES: 2-1, 2-4, 2-6, 2-8, 2-13, 2-16, 2-22, 2-23, and 2-28.

SECTION II

PROGRAM INSTRUCTION CONVENTIONS

2-1 SYMBOLIC ASSEMBLY PROGRAM (SAP).

Several versions of SAP are available to translate symbolic coding into object language acceptable to the Model 980A Computer. Figure 2-1 is a sample source main program, written in symbolic assembly language, ready to be punched into cards or paper tape or otherwise prepared for input. Figure 2-2 is a source subroutine. A source program is input to SAP which generates two outputs. The first is an object program that can be loaded into the computer and executed or linked with other object programs. The object program can be output on punched cards, paper tape, or other media. The second output is an assembly listing as depicted in Figure 2-3 for the main program and Figure 2-4 for the subroutine. Note the following in the output examples:

- a. The items listed under A are an exact reproduction of the handwritten entries on the coding sheet.
- b. The items under B are a hexadecimal representation of the corresponding instructions and constants.
- c. The items under C show the hexadecimal addresses of the instructions, constants, and areas of storage specified by the programmer.
- d. The items under D show the decimal line or sequence number of the source statements to be used in case the program is changed.

SAP is a two-pass assembler, that is, it scans the source program twice. During the first pass, the source program is read and a symbol table is generated. This is accomplished through the use of a location counter in the assembler. The location counter keeps track of the storage locations that will be required by the object program. When a source statement contains a name, the current setting of the location counter is assigned to the name. Each name and the address assigned to it is placed in the assembler's symbol table. During the second pass, the symbol table is used to complete the assembly, and to produce the object with its assembly listing. If bulk storage is available, some versions of SAP copy the source to bulk storage during pass one. Since the output from the first pass is used as input data for the second pass, this eliminates the requirement to manually enter the source data twice.

2-2 SYMBOLIC LINE FORMAT.

The symbolic input line accepted by the assembler may contain a name field, operation field, operand field, and a comment field — or the entire line may be a comment. An input line is the first 64 characters read from a card or in the case of paper tape, an input line is a string of characters the last of which are shown in Figure 2-5. The input line must not exceed 64 characters, not including the end-of-line characters (paper tape only). Input lines are free form within the limits described herein. (Continued on page 2-6)

SYMBOLIC CODING FORM

	5	10	15	20	25	30	35	40	45
			HED	MODEL	980A	MAIN	PROGRAM		
			IDT	ILLUS		6	CHARS.	FOR	OBJECT
			ORG	1000		TELL	SAP	RUN-TIME	
			BRS	1000		ORIGIN	AND	BASE	
BASE			DATA	1000					
			REF	SUB		EXT.	REF.	FOR	LINKING
START			LDA	BASE		ACTUALLY	SET	BASE	
			RMO	0,6		FOR	EXECUTION		
			@BRL	SUB		ADD	2	NOS.	TOGETHER
			DATA	ADDR1		ADDR.OF	FIRST	NO.	
			DATA	ADDR2		ADDR.OF	SECOND	NO.	
			STA	ANSWER		ANSWER	IN	REG.A	
.		.				MORE	EXECUTABLE		
.		.				INSTRUCTIONS	AND		
.		.				ASSEMBLER			
.		.				DIRECTIVES.			
ADDR1			DATA	7		FIRST	NO.		
ADDR2			DATA	8		SECOND	NO.		
ANSWER			BSS	1					
			END	START					

FIGURE 2-1. CODED MAIN PROGRAM

If two commas appear successively, the value of the missing expression is understood to be zero. If fewer than the required number of expressions appear in a source line, the missing expressions are assumed to be zero. If the currency symbol (\$) appears as an element in an expression, the current value of the assembler's location counter is used as its numeric equivalent.

Expressions are strings of items separated by arithmetic operators and terminated by a space, comma, or end-of-line characters. The arithmetic operators are:

Addition	+
Subtraction	-
Multiplication	*
Division	/

If two operators appear in succession, a zero item is assumed.

An item consists of a symbolic address, dollar sign (\$), or a numeric value. If the first character of an item is not numeric, \$, or >, it is assumed to be symbolic. Numeric items may be octal, decimal, or hexadecimal. An octal item is a string of octal characters (0-7), the first of which is zero. A decimal item is a string of numeric characters (0-9), the first of which is non-zero. A hexadecimal item is a greater than symbol (>) followed by a string of hexadecimal digits (0-9, A-F). When using paper tape input, the back slash (\) may be used in place of > to indicate hexadecimal.

Expressions are evaluated left to right using normal arithmetic precedence; i.e., all multiplications and divisions are performed in order of occurrence. The additions and subtractions are then performed in order of occurrence.

All quantities are treated as integers. In division only the quotient is retained and any remainder is discarded. Division by zero is performed as division by one and is not considered as an error.

Sample expressions are:

```
JOE+TOM*3/BOB
$+5
LEA-6
5034
XYZ+>F4
```

NOTE

All expressions are acceptable in absolute assemblies, but multiplication and division involving labels is not allowed in relocatable assemblies. Hence, the first sample would cause a relocation error in a relocatable program.

2-2.5 COMMENT FIELD. Comments may optionally be written on any line. Any characters which appear between the space which terminates the operand field and the end-of-line characters or card column 73 are treated as commentary. The comment field has no effect on the assembly process.

2-3 MACHINE INSTRUCTION GENERAL SYMBOLIC FORMATS.

ADD 2-3.1 REGISTER-MEMORY INSTRUCTIONS. The general symbolic format of a register-memory instruction is:

AND
BIX
BRL
BRU
CPA
CPL
DAD
DIV
DLD
DMT
DSB
DST
IMO
IOR
LDA
LDE
LDM
LDX
MPY
STA
STE
STX
SUB

NAME	OPERATION	OPERAND
LABEL	@OP	=*DISP,MODE

The name field, the asterisk (*) which denotes indirect addressing, the at (@) symbol which denotes extended format, and the equal (=) which denotes immediate operand, are optional. Both the = and * may not be present. The MODE item indicates the addressing mode and may have a value of 0 through 7 (bottom of Table 2-1). It is the value of the IXB bits of the object instruction. DISP and MODE are symbolic expressions. Typical symbolic statements are:

NAME	OPERATION	OPERAND
X	EQU	2
	:	
	:	
PT1	LDA	BUFFER+2
	STA	*ADDR
PT2	LDX	=-5 (-5,7 ALTERNATE OPERAND)
	@STA	ANS,X (ANS,2 ALTERNATE OPERAND)
	BIX	\$-2

NOTE

Instructions DLD, DST, DAD, DSB, with the IXB and displacement field zero cause the program counter to be incremented twice and the two words immediately following the instruction to be treated as data. However, the assembler generates only one word of data following the instruction when using the symbols @DLD, @DST, etc., as with the other extended format register-memory instructions.

Table 2-1 defines the transliteration process performed by the assembler for register-memory instructions. Table 2-1 uses symbolic indicators for the mode expressions. In practice, the user must write EQU (equate) statements for the assembler to interpret any such symbolism, such as that shown for the mode numbers 0 through 7 at the bottom of Table 2-1. Note that it may be preferred to have the label B equated to register number 6; thus, a different label will have to be used for mode B = 1.

In order to translate the operand address expression in register-memory format instructions, the assembler first evaluates the expression as a 16-bit number and then performs one of the following operations:

- A number one greater than the location counter is subtracted to make the address program counter relative.
- A number given by a BRS directive is subtracted to make the address base register relative.

2-4.19 PEJ-PAGE EJECT. The remainder of the current listing page is ejected. The assembler begins a new page headed by the current HED. The PEJ itself is printed as the first line on the new page.

2-4.20 REF-REFERENCED EXTERNAL SYMBOLS. The REF directive identifies a linkage symbol as an external symbol that is referenced in this program. Each such external symbol must be identified in a REF directive. The format of the REF statement is:

NAME	OPERATION	OPERAND
	REF	<i>symbol₁, symbol₂, . . . , symbol_n</i>

The external symbols (separated by commas) in the operand field must be defined in another program and identified in that program as an entry-point symbol by the DEF directive.

As an example, if MTPLY is an entry point symbol in another program the using program identifies it as an external symbol, thus:

OPERATION	OPERAND
REF	MTPLY

The only way an external symbol may be referenced is as a full 16-bit address. The SAP assembler allows an external symbol to be used in an arithmetic calculation. For example, use of MTPLY+2 is allowed.

To link to a program named SINE, the following coding might be used:

NAME	OPERATION	OPERAND
PROGA	BSS	2
	REF	SINE
	.	
	.	
	.	
ADSINE	@BRL	SINE

2-4.21 UNL-STOP LISTING. Suppress assembly listing until the LIS directive is encountered.

2-5 OBJECT FORMAT.

The object program output by the assembler is in the form of standard object records used by all system programs in the Basic System. Information from the IDT and ORG directives is used to generate the header data. If the COMM directive is used, one common area is specified named bBLANK, where b represents an ASCII blank; otherwise, the object contains no common symbols. Entry point records and external reference records are constructed as specified in the DEF and REF directives, respectively. The required text records are created by the assembler, and the end record is generated from the END directive. No block data records are output by the assembler. Details of the object records are covered in the manual, Basic System Use and Operation for the Model 980A Computer.

2-6 SAP ERROR MESSAGES.

The various versions of the assembler may detect certain syntax errors in the source program. When this occurs, a diagnostic message (or the message number on SAP4) is printed in the listing adjacent to the line in question. These comments (listed below) apply only to those versions of SAP which operate in the Model 980A Computer. The IBM System/360 SAP produces error messages which are slightly different. These messages are given in the manual, IBM 360 Support Software for the Model 980A Computer.

Message Number	Message	Meaning (and Corrective Action)
1	FIELD SZ	Address beyond reach (use @ for extended format)
2	UNDF OP	Undefined operation code (check list of valid op codes)
3	LONG SYM	Symbol > 6 characters
4	MDF O/F	OPD or FRM multiply defined (rename label)
5	FRM > 16	FRM fields contain more than 16 bits
6	CAD > 10	Address expression has > 10 elements
7	UNDF SYM	Symbol not defined (label probably omitted)
8	MDF SYM	Symbol multiply defined (rename labels)
9	RELOC	A relocation error (use only one relocatable label in arithmetic expression, or ORG statement can use only one relocatable label)
10	SYM OV	Too many symbols have been defined (cut out symbols or divide program)
11	BAD NUM	Numeric element not valid (properly define item in label or address field)
12	NO END	No END assembler directive found (put in END card before end-of-file)
13	LN > 64	Source line is too long (comment, label, and op code ≤ 64 characters)
14	IMP R/D	A REF or DEF symbol has been used improperly (REF symbol defined inside and outside the program, DEF symbol not defined in the program)
15	X RF USE	A REF symbol has appeared invalidly in an unrelocatable expression
16	IXB ERR	Address mode error (improper use of IXB field (Table 3-1 in ALMI manual))
17	OPD ERR	No such format number (OPD format numbers 0 to 8 explained in paragraph 2-4.17)
18	ADR MODE	Illegal addressing mode (improperly written address)



.

.



.

.



Sales and Service Offices of Texas Instruments are located throughout the United States and in major countries overseas. Contact the Digital Systems Division, Texas Instruments Incorporated, P.O. Box 1444, Houston, Texas 77001, or call (713) 494-5115, for the location of the office nearest to you.



Texas Instruments reserves the right to make changes at any time to improve design and supply the best product possible.

TEXAS INSTRUMENTS
INCORPORATED